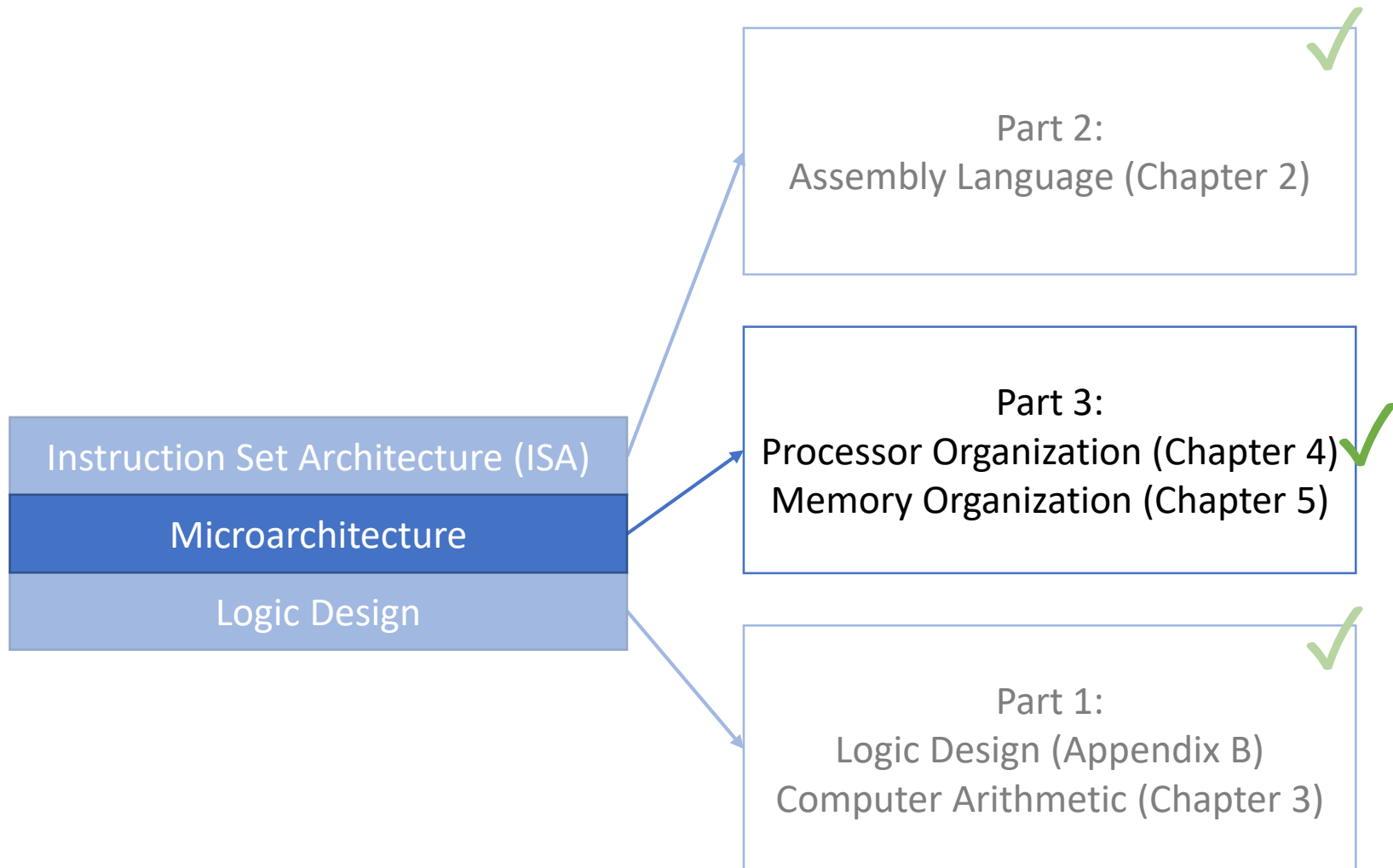


Lecture 28:

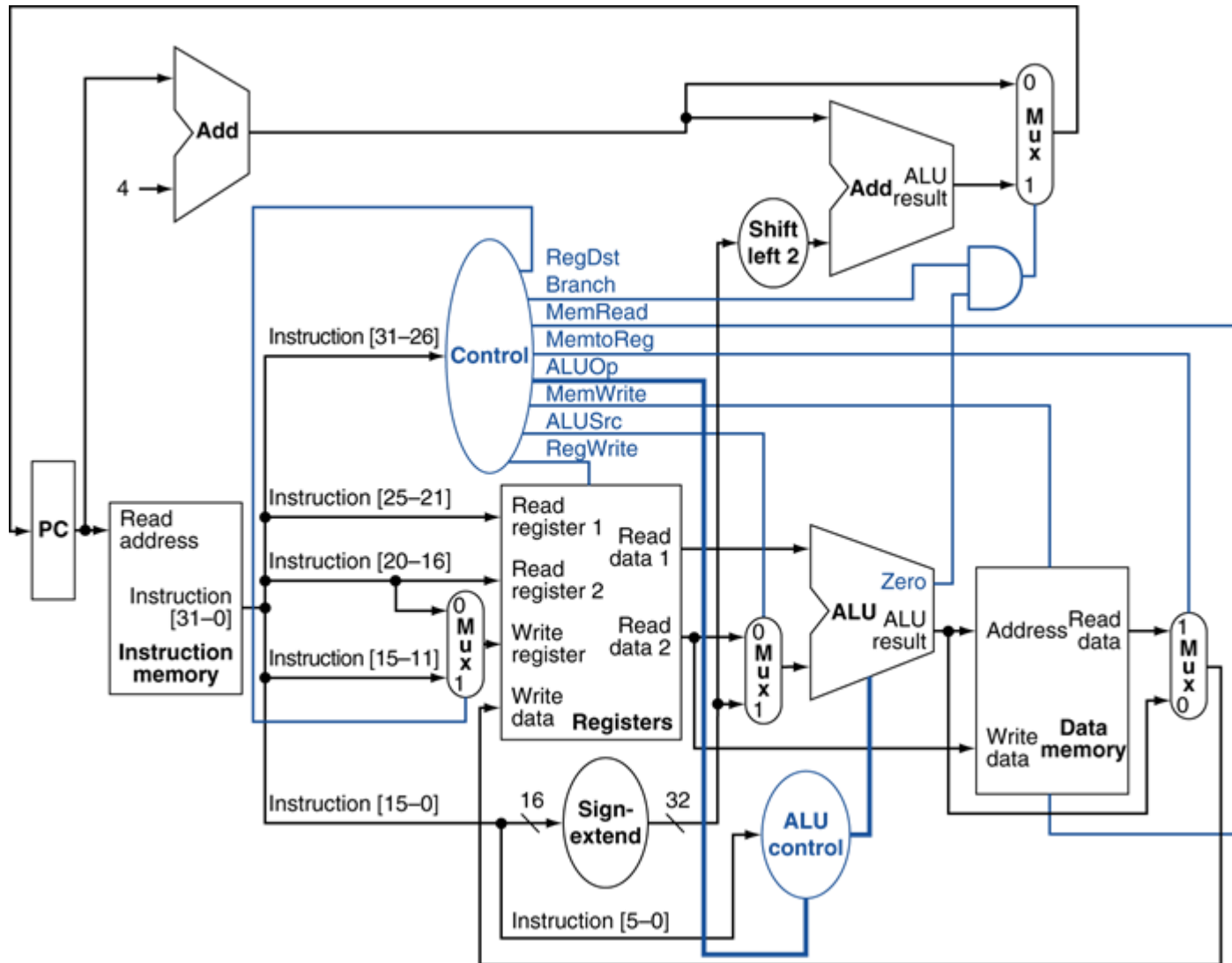
Memory and Cache

CMPS 221 – Computer Organization and Design

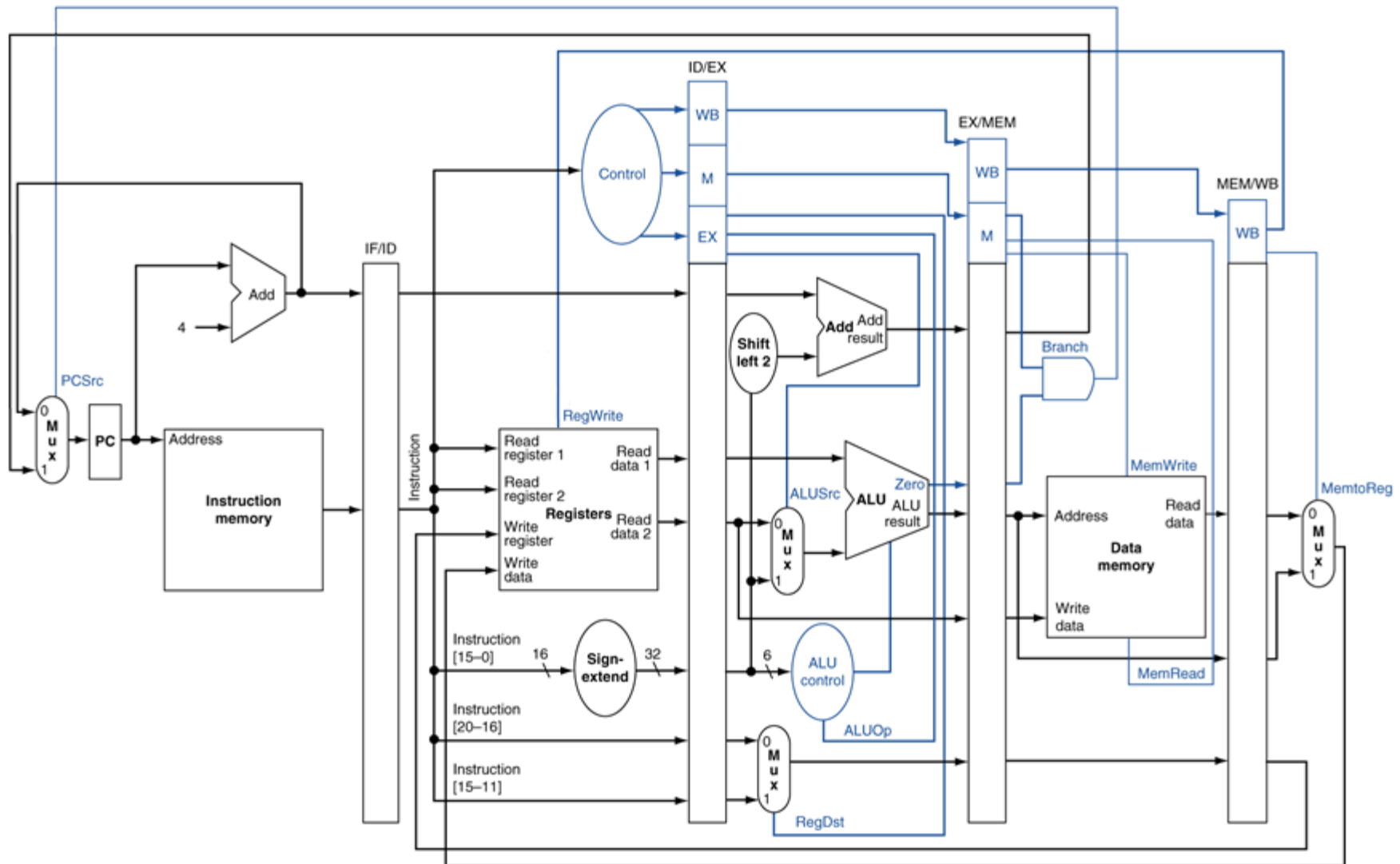
Last Time: End of Processor Organization



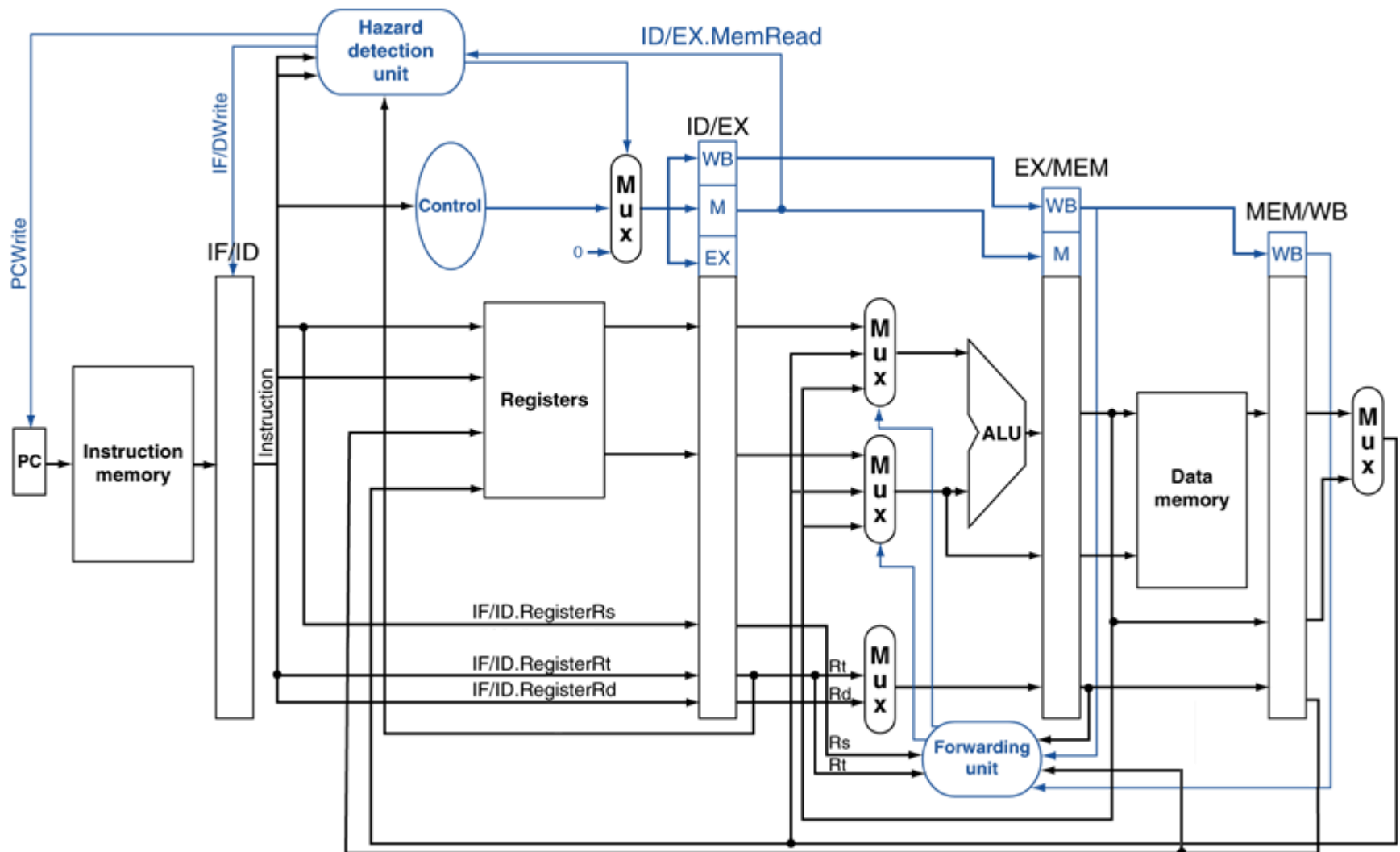
Datapath



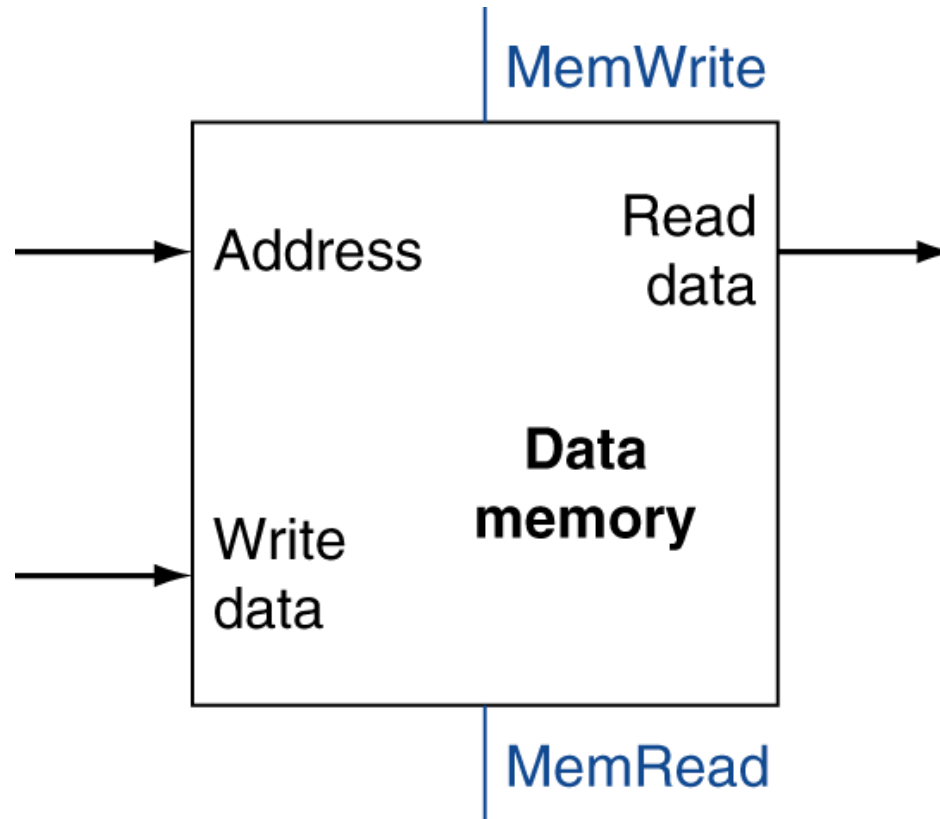
Pipelined Datapath



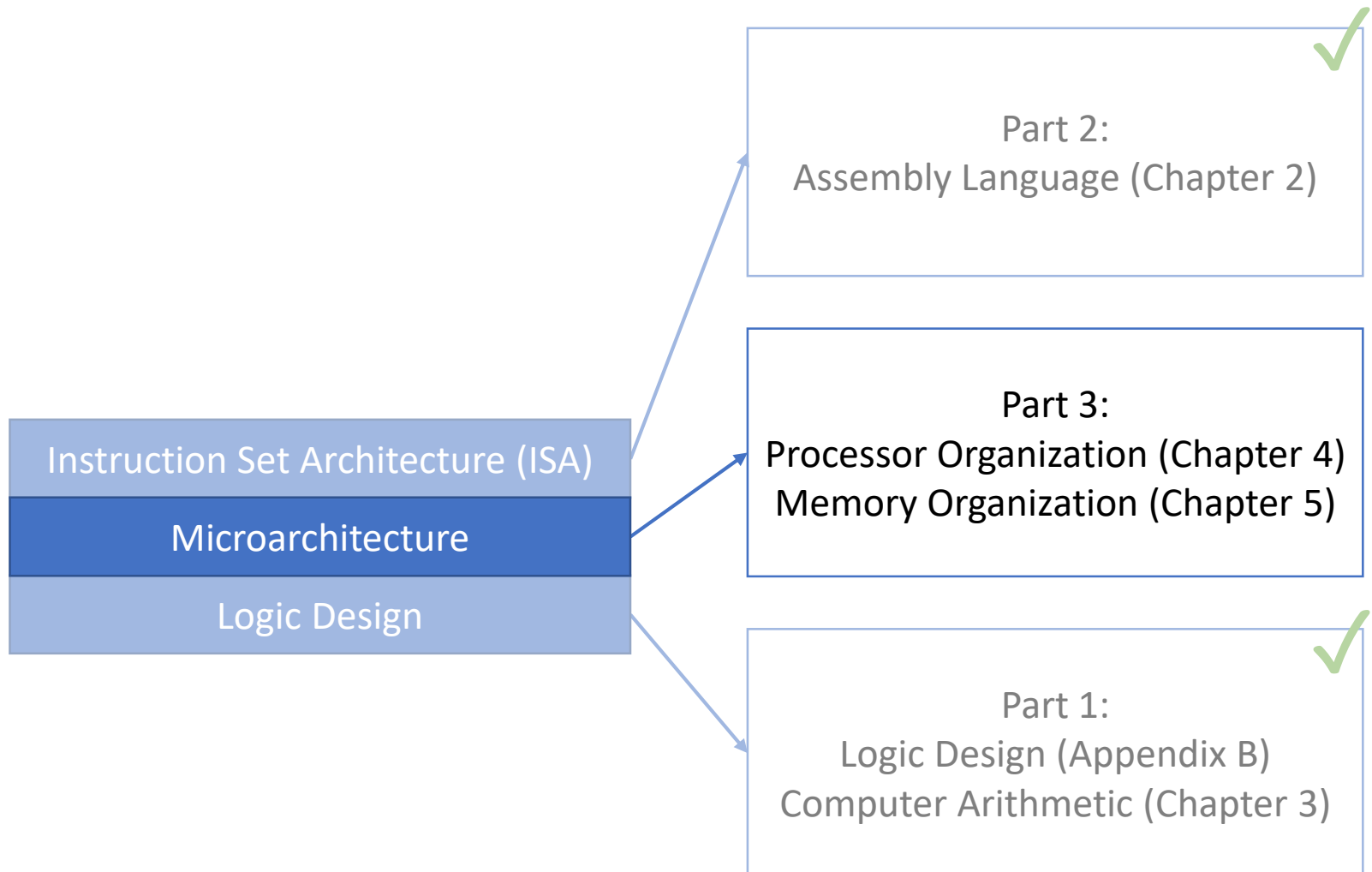
Forwarding and Hazard Detection



Remaining: Memory and Cache



Today: Memory and Cache



Memory Technology

Memory Technology	Access Latency	\$/GiB in 2012
SRAM	0.5-2.5 ns	\$500-\$1,000
DRAM	50-70ns	\$10-\$20
Flash	5,000-50,000 ns	\$0.75-\$1.00
Magnetic Disk	5,000,000-20,000,000 ns	\$0.05-\$0.10

- Ideal memory:
 - Access latency of SRAM
 - Capacity and cost/GiB of disk
- Strategy: arrange memory in a *hierarchy*
 - Smaller and faster memory for data currently being accessed
 - Larger and slower memory for data not currently being accessed

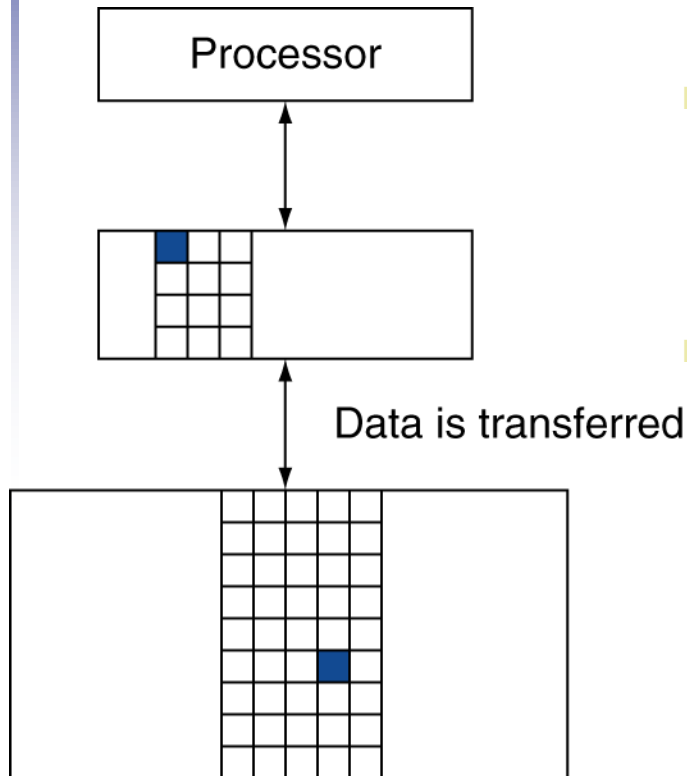
Memory Hierarchy

- Arrange memory in a **hierarchy**
- Store all data on a large flash or magnetic **disk**
- Copy recently accessed (and nearby) items from disk to a smaller DRAM memory
 - Called **main memory**
- Copy more recently accessed (and nearby) items from DRAM to a smaller SRAM
 - Called **cache memory**, closest to the CPU
- “Recently” and “nearby” are good predictors of “currently” because of the *principle of locality*

Principle of Locality

- The **principle of locality** is the observation that programs access a small proportion of their address space at any time
- **Temporal locality**
 - Items accessed recently are likely to be accessed again soon (i.e., locality across time)
 - e.g., instructions in a loop, induction variables
- **Spatial locality**
 - Items nearby those accessed recently are likely to be accessed soon (i.e., locality across space)
 - e.g., sequential instruction access, array data

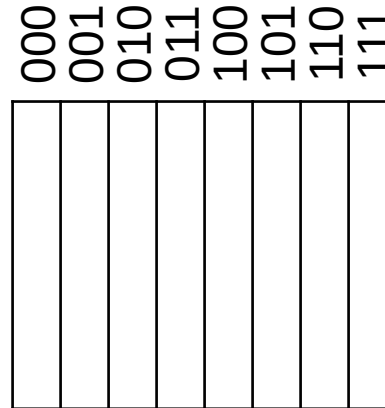
Memory Hierarchy Levels



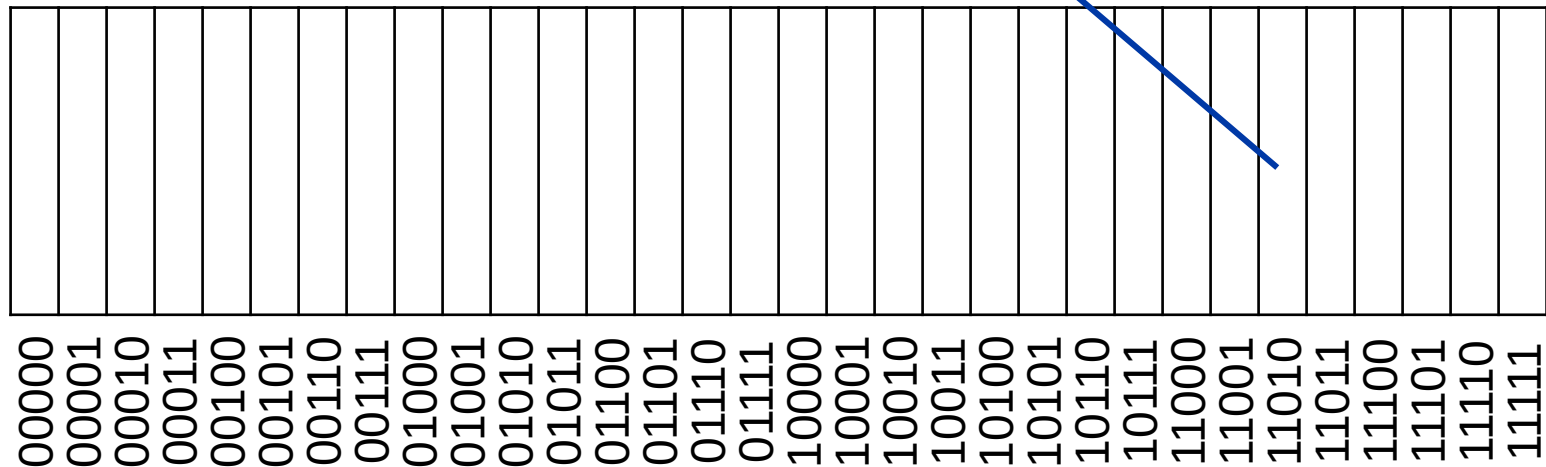
- A **block** (or **line**) is a unit of copying
 - May be multiple words
- A **hit** at a level occurs if accessed data is present at the level
 - **Hit ratio**: hits/accesses
- A **miss** at a level occurs if accessed data is not present at the level and needs to be fetched from the next
 - **Miss ratio**: misses/accesses
 - $\text{miss ratio} = 1 - \text{hit ratio}$
 - **Miss penalty**: time to fetch data from the next level
 - Then accessed data is stored in and supplied from the closest level

Block Placement

Consider a cache
with (2^3) 8 blocks
indexed with 3 bits



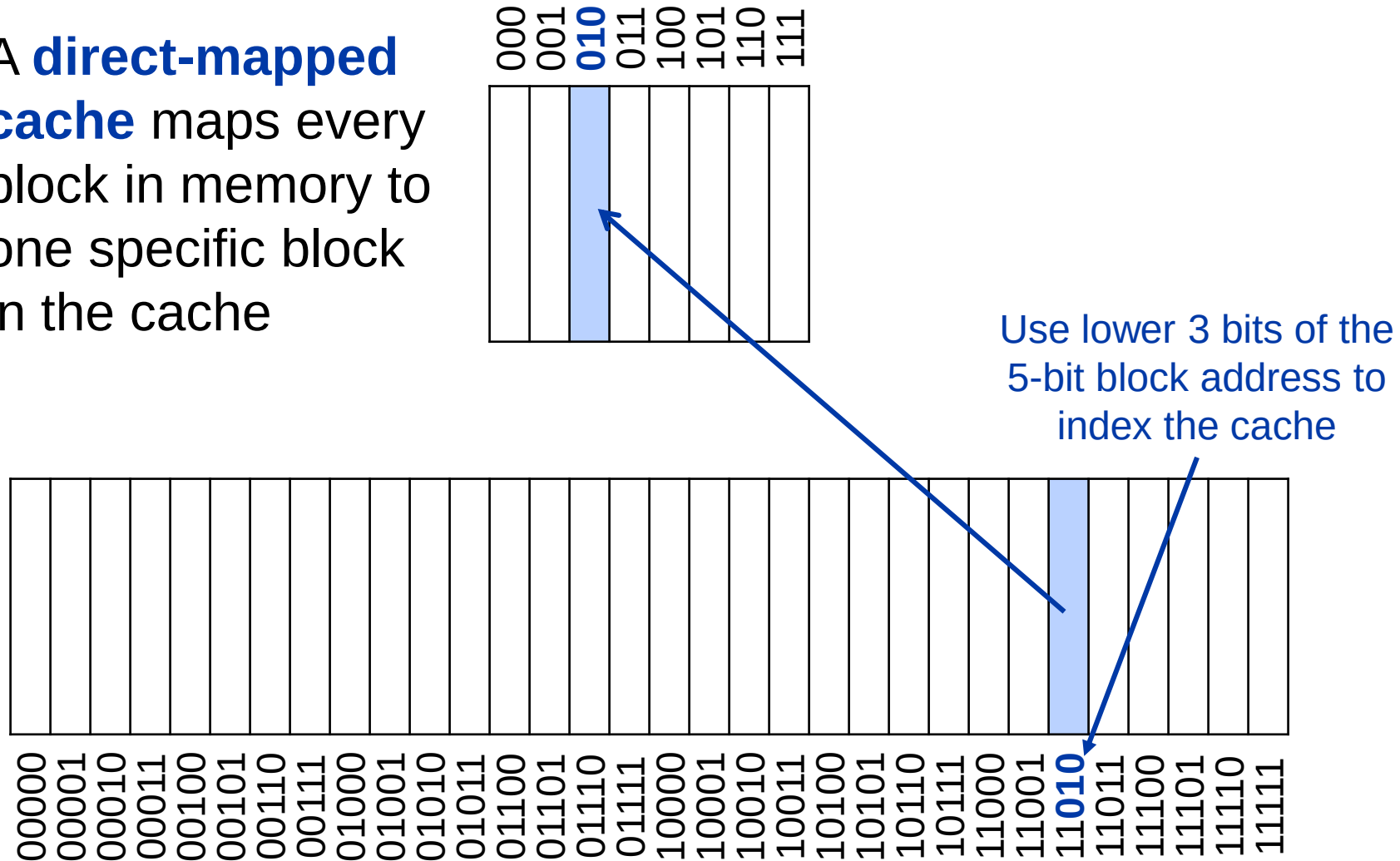
Consider a memory with 2^5 (32) blocks and 5-bit block addresses



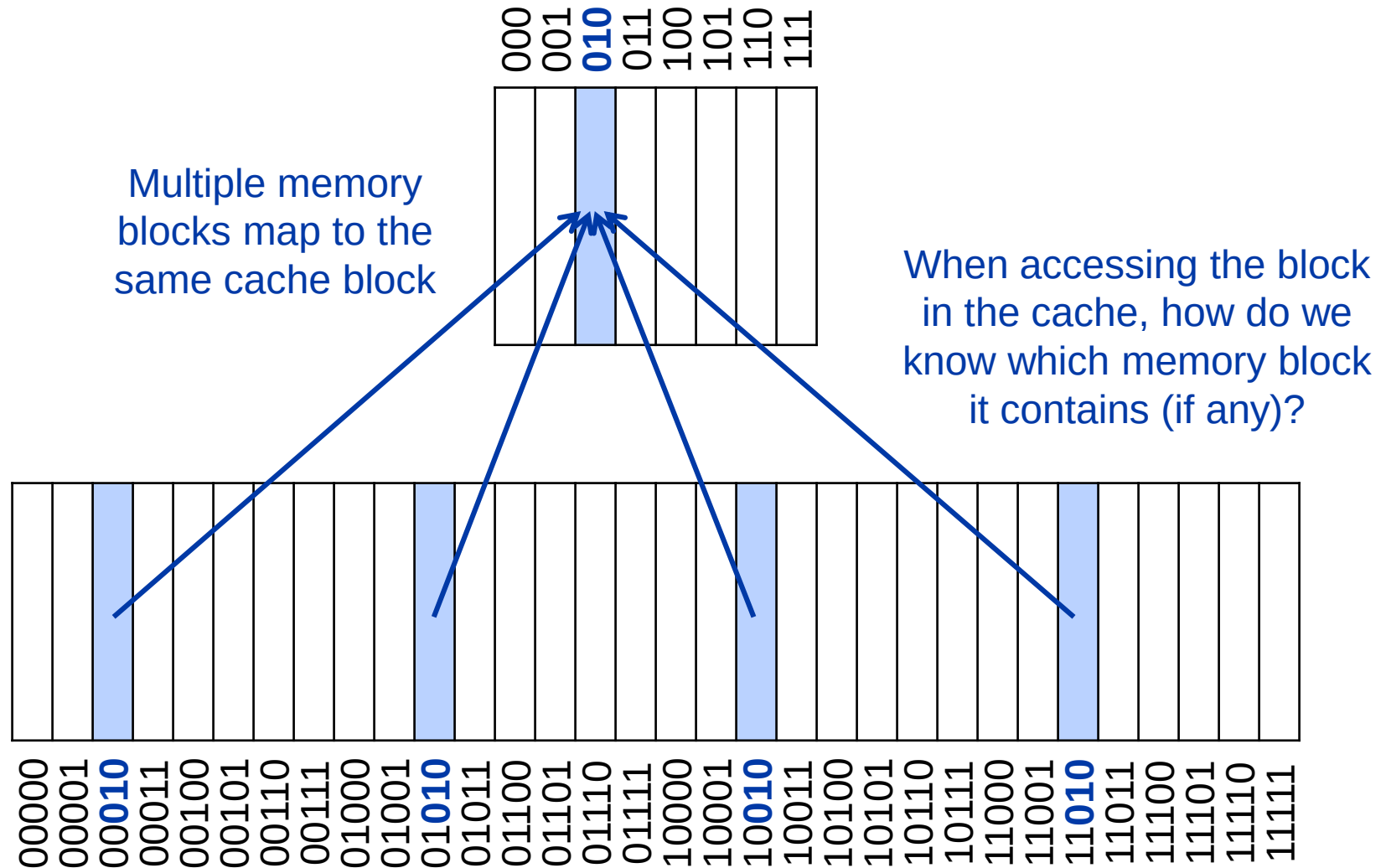
When copying a block from memory to cache, where do we place it?

Direct Mapped Cache

A **direct-mapped cache** maps every block in memory to one specific block in the cache



Direct Mapped Cache



Tags and Valid Bits

- How do we know which memory block is stored in a cache location?
 - Store the address of the block with the data
 - Only higher bits are needed (index is implicit)
 - Called the **tag**
- What if there is no data in a location?
 - **Valid bit**: 1 = present, 0 = not present
 - Initially 0

Cache Example

- Example: direct-mapped cache with 8 blocks

Index	V	Tag	Data
000	0		
001	0		
010	0		
011	0		
100	0		
101	0		
110	0		
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 110		

Index	V	Tag	Data
000	0		
001	0		
010	0		
011	0		
100	0		
101	0		
110	0		
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 110	Miss	110

Index	V	Tag	Data
000	0		
001	0		
010	0		
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
11 010		

Index	V	Tag	Data
000	0		
001	0		
010	0		
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
11 010	Miss	010

Index	V	Tag	Data
000	0		
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 110		

Index	V	Tag	Data
000	0		
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 110	Hit	110

Index	V	Tag	Data
000	0		
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
11 010		

Index	V	Tag	Data
000	0		
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
11 010	Hit	010

Index	V	Tag	Data
000	0		
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 000		

Index	V	Tag	Data
000	0		
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 000	Miss	000

Index	V	Tag	Data
000	1	10	Mem[10000]
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
00 011		

Index	V	Tag	Data
000	1	10	Mem[10000]
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
00 011	Miss	011

Index	V	Tag	Data
000	1	10	Mem[10000]
001	0		
010	1	11	Mem[11010]
011	1	00	Mem[00011]
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 000		

Index	V	Tag	Data
000	1	10	Mem[10000]
001	0		
010	1	11	Mem[11010]
011	1	00	Mem[00011]
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 000	Hit	000

Index	V	Tag	Data
000	1	10	Mem[10000]
001	0		
010	1	11	Mem[11010]
011	1	00	Mem[00011]
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 010		

Index	V	Tag	Data
000	1	10	Mem[10000]
001	0		
010	1	11	Mem[11010]
011	1	00	Mem[00011]
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

Block Address	Hit/miss	Cache block
10 010	Miss	010

Index	V	Tag	Data
000	1	10	Mem[10000]
001	0		
010	1	10	Mem[10010]
011	1	00	Mem[00011]
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

**Evict the
old block**

Address Subdivision

Consider a cache
with 2^{10} (1024) blocks

Index	Valid	Tag	Data
0			
1			
2			
...			
...			
...			
...			
1021			
1022			
1023			

↔
Each block is
1 word (4 bytes)

Address Subdivision

Address (showing bit positions)

31 30 1 0

--

How do we subdivide a 32-bit memory address to access the cache?

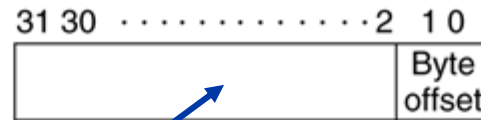
Consider a cache with 2^{10} (1024) blocks

Index	Valid	Tag	Data
0			
1			
2			
...			
...			
...			
...			
1021			
1022			
1023			

↔
Each block is
1 word (4 bytes)

Address Subdivision

Address (showing bit positions)



Remaining 30 bits
are the block address

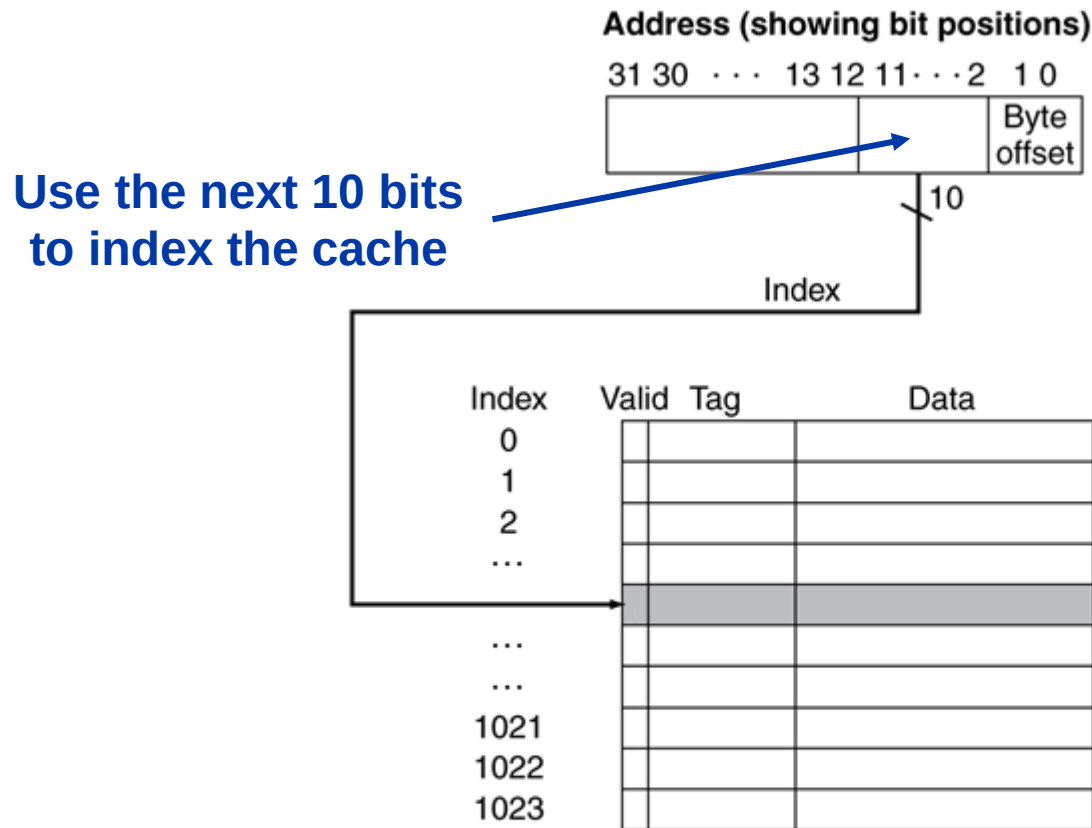
Lower 2 bits are
not needed
(address of a word is a
multiple of 4 ending in 00)

Consider a cache
with 2^{10} (1024) blocks

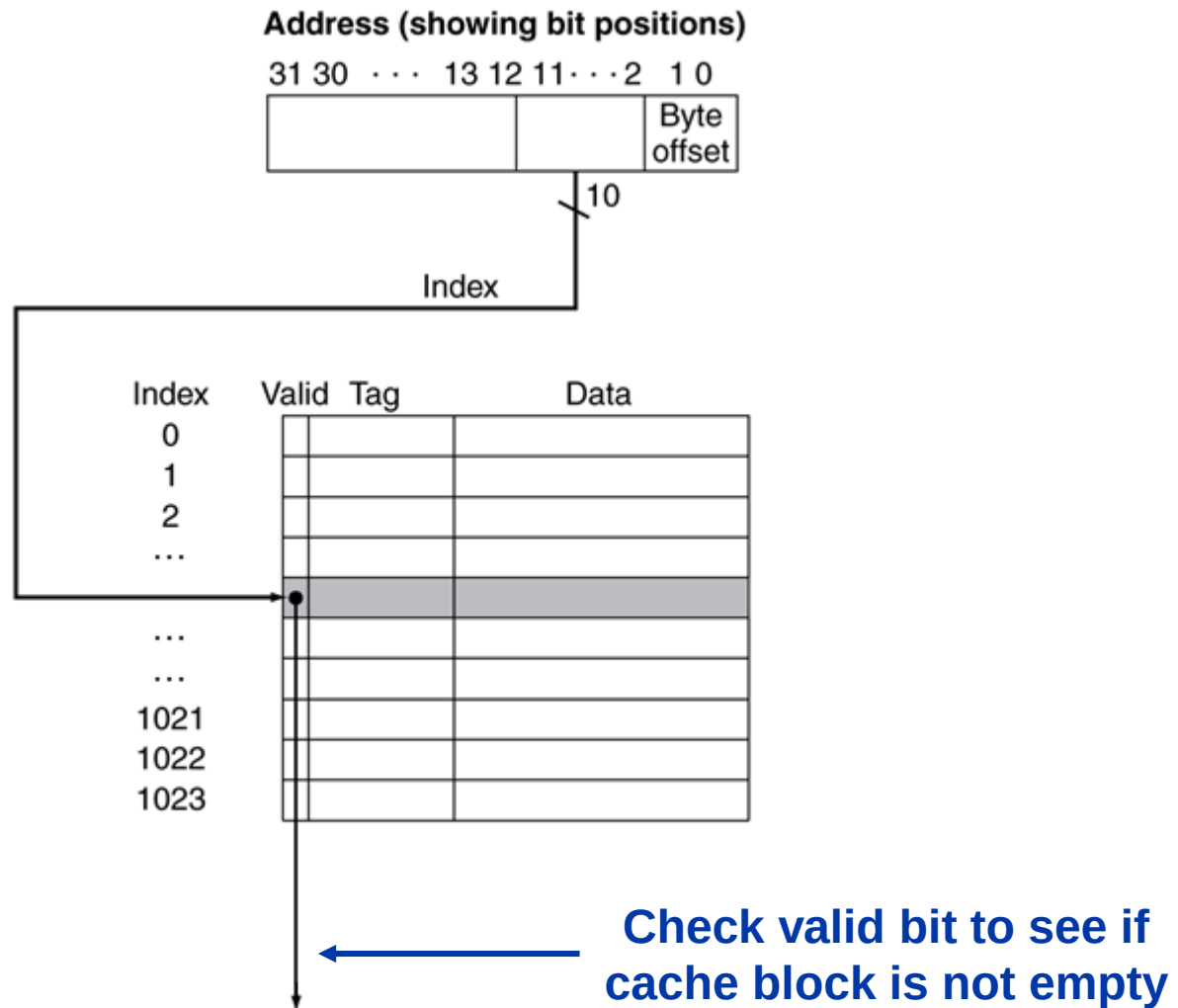
Index	Valid	Tag	Data
0			
1			
2			
...			
...			
...			
1021			
1022			
1023			

Each block is
1 word (4 bytes)

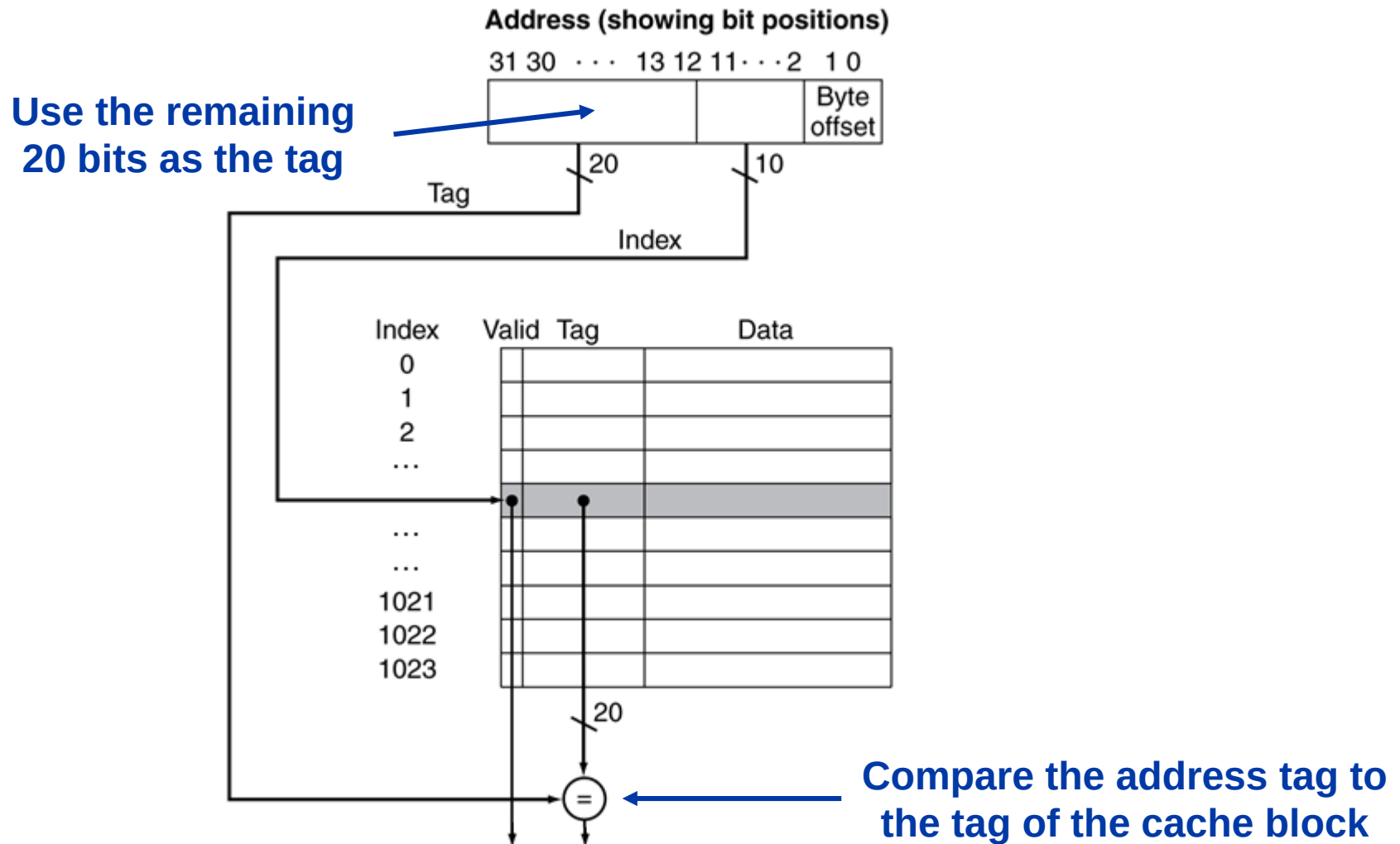
Address Subdivision



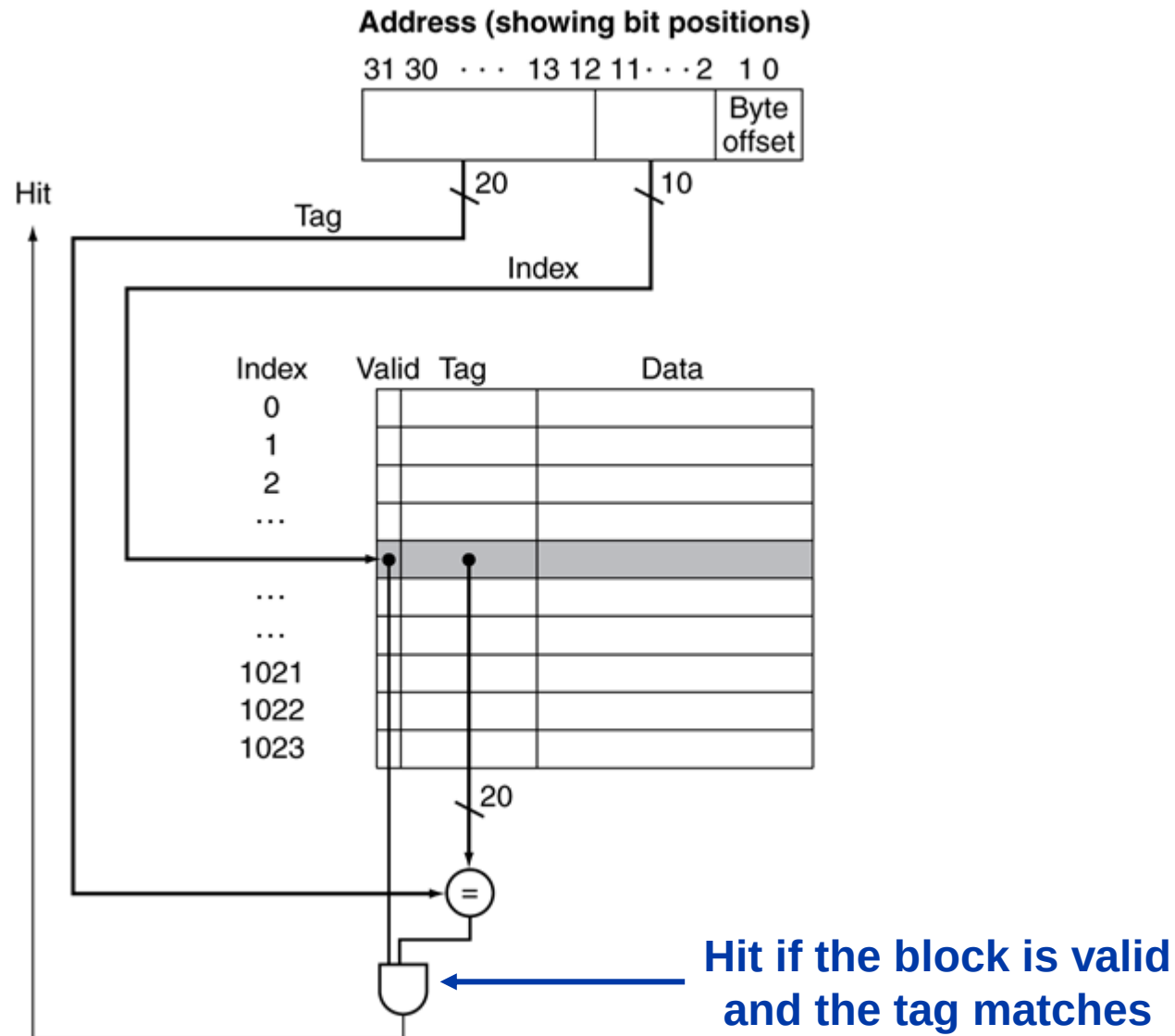
Address Subdivision



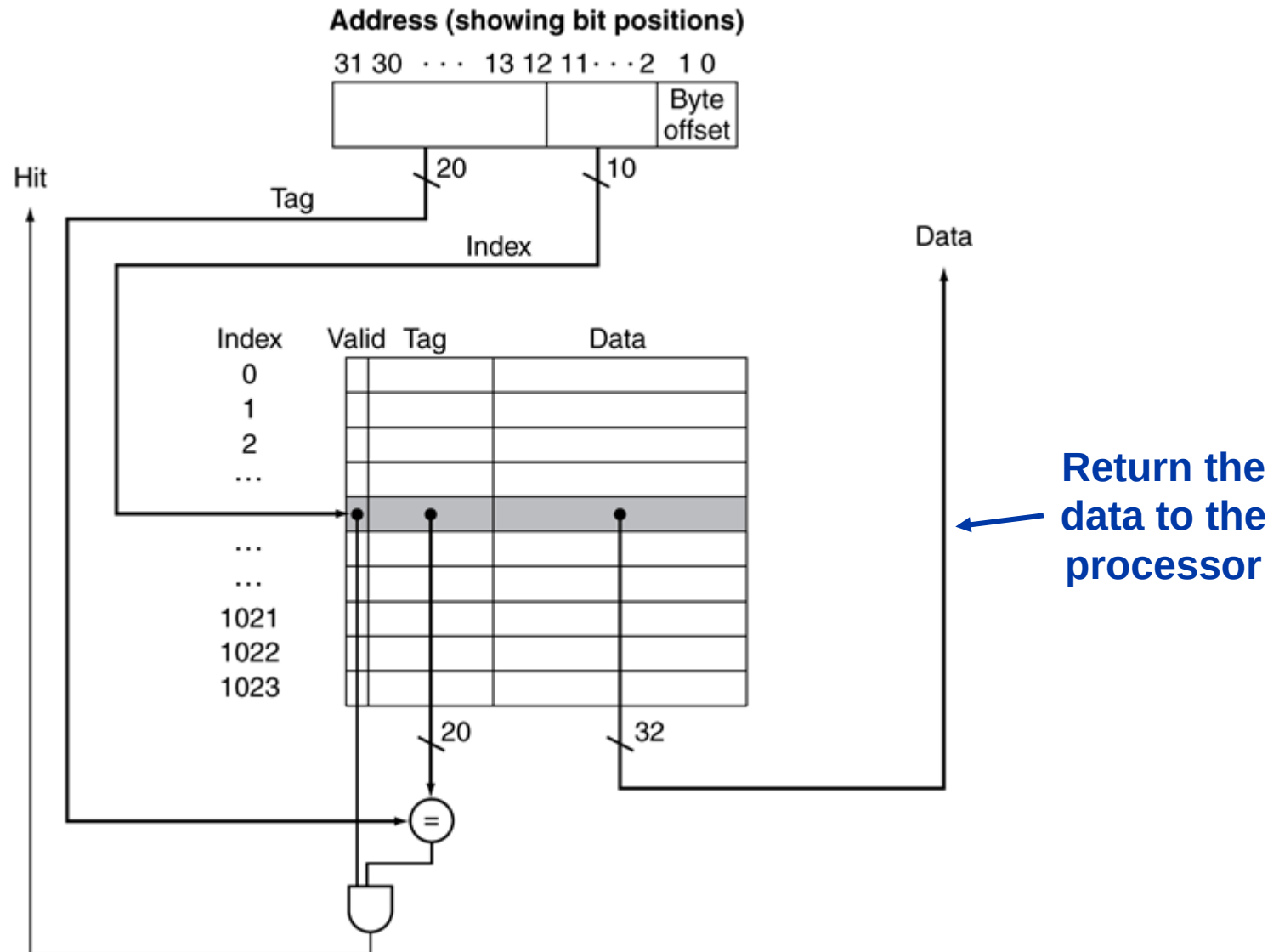
Address Subdivision



Address Subdivision



Address Subdivision



Cache Misses

- On a cache **hit**, the processor proceeds normally
- On a cache **miss**
 - Stall the pipeline
 - Fetch block from next level of hierarchy
 - Resume execution when block arrives
 - Data cache miss
 - Restart memory stage
 - Instruction cache miss
 - Restart instruction fetch stage

Textbook Sections

- The content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - 5.1, 5.2, 5.3